Using Topology-Aware Reinforcement Learning to Synthesize Quantum Linear Reversible Circuits

Giovanni Acampora[1], Allegra Cuzzocrea[1], Marco Lapegna[2], Roberto Schiattarella[1], Autilia Vitiello[1]
[1] *Department of Physics "Ettore Pancini"*, University of Naples Federico II, Naples, Italy
[2] *Department of Mathematics and Applications "Renato Caccioppoli"*, University of Naples Federico II, Naples, Italy
{giovanni.acampora, allegra.cuzzocrea, marco.lapegna, roberto.schiattarella, autilia.vitiello}@unina.it

***Abstract*—In the Noisy Intermediate-Scale Quantum (NISQ) era, efficient quantum circuit synthesis is essential for optimizing limited qubit resources and mitigating noise effects. Quantum Linear Reversible Circuits (QLRCs) are a fundamental class of circuits with applications in quantum compilation or quantum error correction. They are composed exclusively of CX and SWAP gates, which are among the noisiest operations on current quantum hardware. Therefore, their efficient synthesis is pivotal for executing QLRCs on NISQ processors, which have restricted connectivity between qubits. In this direction, we introduce a Reinforcement Learning (RL)-based synthesis approach using Proximal Policy Optimization (PPO) to synthesize QLRCs on generic topologies of up to five qubits. This work extends previous work that did not handle different topologies of quantum processors. Our method dynamically adapts to hardware constraints, learning optimal or near-optimal gate decompositions. We compare our approach against Qiskit's synthesis methods for QLRCs, demonstrating that our RL-based synthesizer consistently achieves lower CX gate depth. These results highlight the potential of RL-driven quantum circuit synthesis as a powerful alternative to traditional heuristic-based techniques, paving the way for more efficient AI-based quantum compilation strategies in the NISQ era.**

***Index Terms*—Quantum Circuit Synthesis, Quantum Computing, Reinforcement Learning, Quantum Compiling**

I. Introduction

Using the principles of quantum mechanics, quantum computers offer the potential to solve problems that are currently intractable for classical computers [1]–[3]. In recent years, the contribution of quantum computing to fields ranging from chemistry [4], [5] and biology [6] to artificial intelligence [7] has been steadily increasing. However, current quantum devices are still in their infancy, and executing quantum algorithms requires several precautions. This is why quantum compilation, also known as quantum transpiling, is a critical process that involves the translation of high-level quantum algorithms into a sequence of quantum gates that can be physically executed on a specific quantum processor. Additionally, the quantum noise inherent in Noisy Intermediate Scale Quantum (NISQ) devices requires the the transpiling procedure also optimizes the final quantum circuits with respect to gate count, depth, and error rates [8]. One of the critical steps of quantum transpiling for the practical realization of quantum algorithms is quantum circuit synthesis. It involves the construction of a quantum circuit that implements a specific quantum operator and can be run on physical quantum hardware. This process requires careful consideration of qubit connectivity, as well as optimization techniques aimed at minimizing resource usage and error rates. As the quantum computing landscape continues to evolve, quantum linear reversible circuits (QLRCs), also known as Linear Function circuits, remain indispensable, providing a versatile and scalable tool for both theoretical and practical advancements in the field. A QLRC can be realized exclusively using CX and SWAP gates. These gates are among the noisiest operations on current quantum hardware. For this reason, their efficient synthesis is fundamental for executing QLRCs on NISQ processors, which have restricted connectivity between qubits. Another reason that confirms the need for QLRCs optimization is that it permits to optimize the blocks Clifford circuits are composed of and improve the execution of this crucial class of circuits. In fact, Clifford operators can be synthesized using blocks of three types of gates: the Hadamard gate, the phase gate S and the CX gate [9]. Since the first two can be synthesized with a small number of gates, the CX gate dominates the gate count [10]. Clifford circuits, known as stabilizer circuits, represent an important class of circuits since they can be simulated in polynomial time on classical probabilistic computers [11]. Recently, an interesting reinforcement learning-based approach for quantum synthesis has been introduced for Linear Function, Clifford and Permutation circuits [12]. In particular, an RL agent learns how to construct quantum circuits by iteratively selecting gates from a given set and adding them to the circuit, thereby processing the circuit and receiving feedback in the form of a new state and a reward. Success is achieved when the constructed circuit exactly implements the target operator. However, this method is designed for use with a specific quantum processor topology, requiring the RL agent to be retrained each time a new processor coupling map is used. More recent work fills this gap for Clifford circuits by providing a RL-based model capable of synthesizing Clifford circuits on a wide range of architectures [13]. This approach provides near-optimal circuits in terms of depth and CX depth thanks to the use of the invalid action masking technique [14].

In this paper, a reinforcement learning (RL)-based synthesis approach is introduced for QLRCs specifically. It is able to synthesize Linear Function circuits of up to five qubits on

This work was supported by the National Center for HPC, Big Data, and Quantum Computing (Spoke 1: Future HPC) in the framework of the Next Generation EU program. Giovanni Acampora acknowledges the financial support from the PNRR MUR project PE0000013-FAIR.

generic topologies. This work extends previous work [12] that did not handle different topologies of quantum processors. More specifically, the adaptation of the Clifford RL-based synthesis approach introduced in [13] has been carried out for the case of QLRCs, providing models that respect given hardware constraints and achieve near-optimal performance that surpasses that of the Qiskit's synthesis methods for QLRCs.

The next sections of the paper are organized as follows. Section II introduces the fundamentals of quantum computing and the basic concepts addressed in the context of this work. Section III provides an overview of the state-of-the-art in quantum circuit synthesis. Section IV reports all details about the proposal and section V presents the conducted experiments and a discussion about the results.

II. RELATED WORK

Quantum compiling is an increasingly explored field focused on adapting a quantum circuit to be compatible with a specific architecture, while also optimizing it to minimize the impact of quantum noise. A central area of research in this field is the synthesis of quantum circuits. Various methodologies have been suggested to develop efficient synthesizers. An interesting but quite slow topology-aware synthesis process was introduced in [15] and extended to improve its speed in [16], [17]. Furthermore, in [18] a genetic algorithm was proposed to address this task. However, these approaches do not address different topologies. More recently, artificial intelligence has demonstrated to be a valid alternative synthesis approach that reduces the excessive run times of the above algorithms [19]. In particular, reinforcement learning (RL), a particular branch of machine learning that is increasingly establishing itself as a valuable tool in quantum computing [20]–[23], is gaining particular traction as a promising approach in the field of quantum synthesis. Unlike supervised learning, where the model is trained on labeled data, RL involves trial-and-error learning, where an agent takes actions, receives feedback in the form of rewards or penalties, and adjusts its behavior accordingly to maximize the cumulative reward over time. Initially, the potential of RL-based synthesizers has been explored, concentrating on single-qubit quantum state preparation and gate compilation [24]–[26]. Then, deep Q-learning and A* search have been combined for two-qubit unitaries synthesis [27]. However, the search complexity still grows exponentially. Other RL-based approaches have been proposed but do not consider any topology constraints or are specialized on a single architecture [28]–[32]. Many studies have been conducted on Clifford synthesis, ranging from greedy approaches [9], [33]–[35] to machine learning techniques such as reinforcement learning [12], [13]. Also, quantum linear reversible circuits (QLRCs) synthesis has been investigated. This particular class of circuits plays a fundamental role in quantum computing, serving as essential building blocks for various quantum algorithms and protocols. These circuits, which implement linear transformations over quantum states, are key to efficient quantum computation and are particularly useful in areas like quantum error correction, quantum simulation, and data encoding. The reversibility of these circuits ensures that no information is lost during computation, which is a crucial property for maintaining quantum coherence and preventing errors. Gaussian elimination and LU decomposition are two well-known mathematical techniques commonly used in the synthesis of QCRCs. In quantum computing, Gaussian elimination can be used to determine the optimal sequence of quantum gates that perform specific linear transformations. By converting the matrix representing the circuit's desired transformation into a simpler form, it becomes easier to identify the necessary operations for circuit construction. On the other hand, LU decomposition is beneficial for the synthesis of QLRCs because it enables the separation of a complex linear transformation into simpler, manageable components. Both methods, Gaussian elimination and LU decomposition, play an essential role in optimizing the synthesis of QLRCs by reducing the complexity of the required transformations but they provide circuits with $O(n^2)$ gates in the worst case. Instead, it has been shown that the best lower bound is $O(\frac{n^2}{\log(n)})$ [36]. For this reason, a subsequently proposed work has presented a faster synthesis algorithm that meets the lower bound [10]. It decomposes a binary matrix with asymptotic efficiency. Another work has realized a linear reversible circuit synthesizer for linear nearest-neighbor architectures [37]. This algorithm synthesizes any linear reversible circuit of N qubits over a linear nearest-neighbor architecture using CX gates with depth at most $5N$. These last two approaches are present in Qiskit' SDK as Linear Function synthesis methods. Building on the literature reviewed above, the proposed work is the first to apply a RL-based approach to synthesize QLRCs that maps circuits on a specified topology.

III. BACKGROUND

This section introduces the key concepts of quantum computing and quantum transpiling, with a particular emphasis on the synthesis problem.

A. Quantum Linear Reversible Circuits

A logic gate with n inputs and m outputs can be represented as a function as follows:

$$f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \tag{1}$$

where $\mathbb{F}_2$ represents the two-element field and $\mathbb{F}_2^n$ is the set of all possible n-dimensional vectors over this field. The gate described is reversible if f is bijective, meaning that the inputs can be recovered by the outputs and each possible output can be produced [1]. A notable example of a quantum reversible gate is the CX gate, as it defines a function that maps each input vector to a unique output vector, with all possible outputs attainable. A quantum reversible circuit is a circuit composed of quantum reversible gates only and that computes a bijective function. Furthermore, a circuit is linear if the following equation is satisfied:

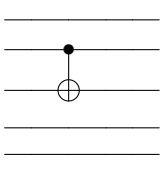
$$f(x_1 \oplus x_2) = f(x_1) \oplus f(x_2) \quad x_1, x_2 \in \mathbb{F}_2^n \tag{2}$$

where $\oplus$ is the bitwise XOR operation.

A quantum linear reversible circuit (QLRC) on n wires represents a linear transformation over $\mathbb{F}_2^n$. Therefore, a circuit like this can be represented as $n \times n$ matrix M such that, given n-dimensional input x, the n-dimensional output y of the circuit is given by the following operation:

$$Mx = y \tag{3}$$

The matrix M is constructed from the truth table of the corresponding circuit meaning that its columns are the outputs that the circuit provides for each input state that is in the basis set of $\mathbb{F}_2^n$. Therefore, the matrix associated with a CX gate with control i and target j is the identity matrix with a further 1 value in the off-diagonal entry placed in correspondence with the i-th column and j-th row. As an example, the matrix representation of a CX gate applied to qubits 1 and 2 of a circuit of 5 qubits is shown below:



$$\Longrightarrow \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This elementary representation that does not scale exponentially with the number of qubits in the circuits offers relevant advantages as it allows to efficiently simulate the circuit's behavior, particularly when dealing with large-scale circuits. This representation is also helpful for optimizing quantum circuits by manipulating the matrices to reduce quantum gate counts or mitigate noise effects.

IV. RL-based Synthesis Process

The execution of quantum circuits on real quantum hardware must be preceded by a compilation process, called transpiling. In fact, each NISQ device is characterized by a specific gate set and a topology that imposes some connectivity constraints. This means that the execution of a quantum circuit on a specific device implies the translation of incompatible gates into compatible ones while using two-qubit gates only for the permitted connections introduced by the topology. Furthermore, the presence of quantum noise in NISQ architectures requires optimal circuits in terms of the number of gates and depth, taking into account that two-qubit operations are much more expensive than single-qubit ones. Quantum synthesis represents a valid approach to minimize the number of two-qubit gates. Formally, given a quantum operator $\mathcal{U}$ and a set of gates $\mathcal{S}$, synthesis involves the construction of a quantum circuit $\mathcal{QC}$ using the gates of $\mathcal{S}$ that effectively implements the operator $\mathcal{U}$.

To perform quantum synthesis via RL, it is important to note that this problem can be tackled as a sequential decision problem, as introduced in [12]. More specifically, an agent learns to make decisions by interacting with an environment to maximize a cumulative reward. Let us consider a target operator $\mathcal{U}$ obtained from a quantum circuit $\mathcal{QC}$. The initial state of the process is represented by the matrix representation of $\mathcal{U}$, $\mathcal{M}$. The state is provided to the agent $\mathcal{A}$ that chooses an action from the action space ($\mathcal{AS}$) which is composed of all possible operations determined by a specific gate set $\mathcal{S}$ and a specified topology $\mathcal{P}$. The action is applied to $\mathcal{QC}$ obtaining a new circuit and consequently a new operator. This represents the next state that is provided to $\mathcal{A}$ together with a reward. The process is iterated until the state reaches the identity matrix, meaning that $\mathcal{A}$ has exactly implemented U. The resulting synthesized circuit is given by the inverse sequence of the gates chosen by $\mathcal{A}$. The pseudocode of the synthesis process is shown in Algorithm 1.

Algorithm 1 RL-based Quantum Synthesis for QLRCs

Require: Topology $\mathcal{P}$, Gate set $\mathcal{S}$, target operator matrix $\mathcal{M}$, trained Agent $\mathcal{A}$, Empty quantum circuit $\mathcal{QC}$, Maximum number of gate in $\mathcal{QC}$;
Ensure: Optimized synthesized circuit $\mathcal{QC}_S$

```
n = 0
while n ≤ Maximum number of gates in QC do
    a ← A.predict(M, S, P)        // Agent predicts action based on current operator according to S and P
    QC ← QC.insert(a)             // Append the gate selected to the circuit
    M ← update(M, a)              // Update the matrix according the action taken
    n = n + 1
    if M = Identity Matrix then
        break                     // Terminate if the current operator is equal to the identity matrix
    end if
end while
QC_S ← reverse(QC) // Invert the order of gates in QC
return QC_S
```

In [12], this process is thought to be executed for a certain number of qubits and a topology of interest, requiring new training each time the architecture changes. For this reason, in [13], the invalid action masking technique is introduced to overcome this limitation, and a generalized approach is presented. In particular, the new approach permits the synthesis of circuits among different topologies simultaneously, making it possible to use a unique model for different architectures. In particular, this technique is used to hide some actions of $\mathcal{AS}$ from $\mathcal{A}$ forcing it to choose from a specific subset $\mathcal{AS}^{\mathcal{P}}$ composed of only gates that respect the topology of $\mathcal{P}$. The validity of the new RL-based approach has been demonstrated by the experimental session, which reports circuits with lower depths and CX depths with respect to state-of-the-art methods, highlighting the power of invalid action masking. However, the introduced environment has been constructed only for Clifford operators. QLRCs constitute the most relevant part of Clifford circuits, demonstrating the cruciality of their optimization. Therefore, an efficient optimization of QLRCs can improve the depth of Clifford circuits. For this reason, in this paper, the described approach for Clifford operators has been adapted

to the QLRCs class, analyzing and validating the generality of the approach in [13] across different classes of quantum operators. In the next section, more specific details about the environment setup and training will be provided.

A. Training process

The Proximal Policy Optimization (PPO) algorithm was adopted for the policy, as used in both [12] and [13]. PPO is an on-policy algorithm that gathers data by interacting with the environment and refines the policy using a surrogate objective function through stochastic gradient ascent. Specifically, the policy gets updated based on a rollout, which refers to the process of using the current policy to interact with the environment and generating a series of state-action-reward sequences, called experiences. To accelerate data collection during rollout, multiple vectorized environments were employed, where multiple copies of the environment are created and run in parallel, allowing for an increase in training speed. In the case of QLRCs, the observation space, the action space, and the reward are defined as follows:

- **Observation space**: it is composed of $N \times N$ boolean matrices as QLRCs operators can be represented in this form.
- **Action space** it consists of the gates that characterize QLRCs. In particular, it can contain either only CX gates or a combination of CX and SWAP gates. It is important to note that a SWAP gate is composed of three CX gates. Also in this case, the invalid action masking technique was adopted in order to cancel the probabilities of invalid actions.
- **Reward**: in both scenarios related to the action spaces and throughout all training phases, the same reward function used in [13] was adopted. It is reported below:

$$\mathcal{R} = \begin{cases} 1000 - \\ D_i(D_i - D_{i-1}) - c_0 Tq_i & \text{if Success} \\ \\ \mathcal{ID} - c_1(D_i - D_{i-1}) - \\ c_2(Tq_i - Tq_{i-1}) - & \text{if not Success} \\ c_3(Sq_i - Sq_{i-1}) \end{cases} \tag{4}$$

where D_i is the depth of the synthesized circuit after applying the i-th action; Tq_i is the number of two-qubit gates and Sq_i is the number of single-qubit gates; c_0, c_1, c_2, and c_3 are coefficients; $\mathcal{ID}$ represents the distance between the matrix M and the identity matrix $N \times N$, after the i-th action. The expression of $\mathcal{ID}$ is as follows:

$$\mathcal{ID} = 1 - \frac{\sum_{lm} I_{lm} - M_{lm}}{N \times N} \tag{5}$$

where I_{lm} and M_{lm} are the values of the identity matrix and M in correspondence with the respective l-row and m-column. The reward function is designed to assign a high positive value when A successfully completes the synthesis process. This value increases as the circuit depth and the number of two-qubit gates are minimized. It is worth observing that differently from the Clifford case, for Linear Function operators there is no phase factor to correct after the synthesis with an RL-based model. This means that the inverse sequence of the circuit obtained from $\mathcal{A}$ directly represents the final result.

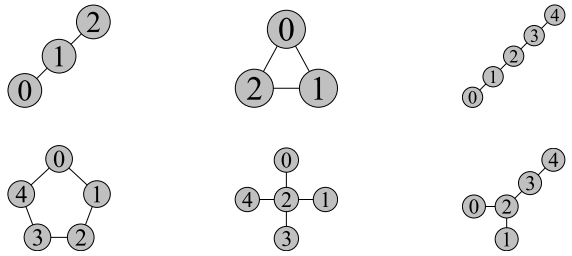
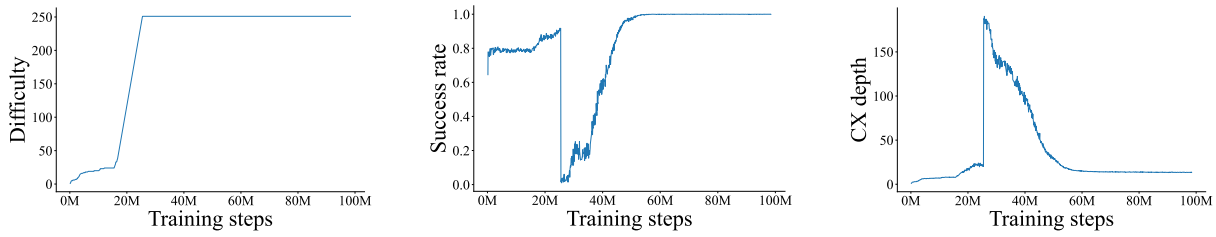


Fig. 1. 3-qubit and 5-qubit topologies used during the training.

As in [13] and [12], the learning process is based on a curriculum learning strategy. In the first phase, target inputs are obtained by generating Linear Function operators from N-qubit circuits composed of d gates uniformly sampled from $\mathcal{AS}^{\mathcal{P}}$, where d represents a difficulty parameter that starts from 1 and increases each time the environments reach a success rate greater than a certain threshold, and $\mathcal{P}$ is randomly chosen from a set of given topologies. The success rate is defined as the ratio of successful episodes to the total number of episodes in a rollout. Each new target operator marks the beginning of a so-called episode which concludes either upon achieving success or when a number of steps equal to d is achieved. In this way, the depth of the resulting circuit is minimized. The second phase of the training starts when d reaches the maximum value set. This second phase is different if the original action space $\mathcal{AS}$ was selected to be composed of both CX and SWAP gates or just CX gates. Indeed, in the former scenario, the difficulty is kept constant at d and the generation of the target operators is similar to the one described in phase 1. In the latter scenario, the target operators that must be synthesized are still kept to difficulty d, but now they can be composed of also SWAP gates. This aims to enhance the capability of the agent to synthesize SWAP gates using only combinations of CX gates.

V. Experiments and Results

In this section, the training and testing procedures for 3-qubit and 5-qubit Linear Function circuit synthesis are explained. Firstly, the details about the training setup are reported. Then, the evaluation of RL-based models with different action spaces is discussed, and subsequently the RL-based synthesizers are compared with the two Qiskit's synthesis methods for QLRCs. The first Qiskit's synthesizer is an implementation of the Patel, Markov, and Hayes (PMH) algorithm from [10] for optimal synthesis of QLRCs for all-to-all architectures, while the second is specialized for linear nearest-neighbor architectures using the Kutin, Moulton, and Smithline (KMS) method [37]. Both algorithms do not handle different architectures, so they are followed by routing with Qiskit SDK's SABRE [38].



(a) (b) (c)

Fig. 2. Training progress related to the training of the 5-qubit Linear Function synthesis model trained using an action space composed of only CX gates. The progression of the quantities of interest over the training steps is presented. In particular, Fig. (a) reports the training difficulty, Fig. (b) the success rate, and Fig. (c) the CX depth.

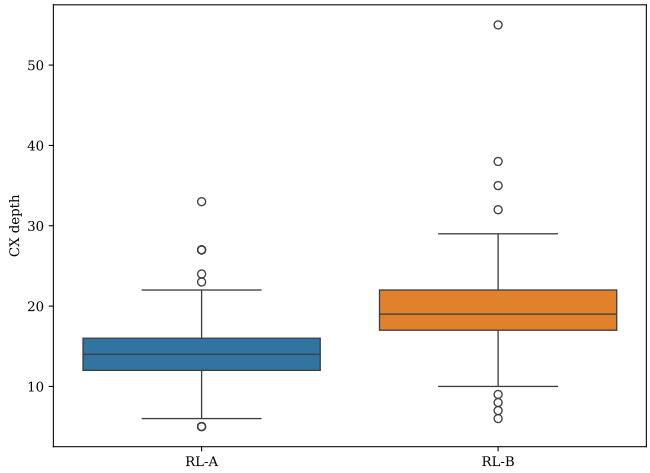


Fig. 3. Comparison between the performance of RL-A and RL-B using all topologies the models were trained on.

A. Training Setup

The environment was implemented using the Gymnasium interface [39]. This interface permitted to accelerate the training creating 40 vectorized environments running in parallel. Furthermore, the multilayer perceptron (MLP) default policy in the MaskablePPO class implemented in Stable Baselines3 [40] was used as the policy of $\mathcal{A}$. The topologies considered during the training for the 3-qubit and 5-qubit cases are reported in Fig. 1. Moreover, the threshold used to increase d was established at 0.8, indicating that the difficulty level increases only when the number of successful episodes over the total number of episodes across all environments in a rollout reaches 80%. The maximum value for d was set to 250, which means that the model begins phase 2 of its training when it can synthesize operators up to 250 gates. To provide an overview of the training process, Fig. 2 shows the progress for the 5-qubit case. In particular, the reported curves refer to an agent trained with $\mathcal{AS} = \{CX_{ij}\}$, where $i, j \in [1, N]$, $i \neq j$. It can be seen that in a first stage, the success rate increases up to the predefined threshold, meaning that the model is learning to synthesize operators associated with circuits with one gate. Subsequently, the curve stabilizes around the threshold before gradually increasing to almost reach a success rate of 100%. Once the maximum difficulty is reached, the second phase of the training process starts by introducing SWAP gates into the circuits from which operators are generated. For this reason, it can be seen that the success rate decreases sharply before rising quite quickly to 1, meaning that it has learned to synthesize any Linear Function operator up to a difficulty of 250. Finally, the CX depth of the synthesized circuits shows that when SWAP gates are introduced, the CX depth increases drastically before again decreasing and improving performance. This can be explained by considering that a SWAP gate is composed of three CX gates, and the action space is composed only of CX gates. Therefore, an immediate decrease in performance is legitimate. However, the agent rapidly learns to handle SWAP gates and the final model returns to use a low number of CX gates for the synthesis. Finally, the coefficients of the reward function were set as follows: $c_0 = 10$, $c_1 = 0.5$, $c_2 = 0.15$, $c_3 = 0.1$.

B. Analysis of the performance with different action spaces

In this section, it is presented the comparison of two RL-based agents, namely RL-A and RL-B, that have been trained as discussed in sec. V-A. Both the models work for 5-qubit QLRCs, and they differ in their action spaces. In detail, the action space for RL-A ($\mathcal{AS}_A$) is composed of just CX gates, while for RL-B the action space ($\mathcal{AS}_B$) is also equipped with SWAP gates. The goal of this comparison is to understand if a model trained directly to insert only CX gates results in shallower synthesized QLRCs with respect to one that directly adds SWAP gates. In other words, we are trying to understand if the RL agent trained to add just CX gates is able to find patterns that further optimize the structure of a QLRC. The experiments carried out to perform such a comparison, consist of synthesizing both by RL-A and RL-B 200 QLRCs each of which on a randomly selected 5-qubit topology from those reported in Fig. 1. In detail, the sizes of the original target quantum circuits were randomly selected between 1 and 1000 quantum gates. Both the models were able to synthesize 100% of the test circuits. This demonstrates that training up to a difficulty level of 250 was sufficient to enable the model to synthesize circuits with significantly higher complexity. Fig.

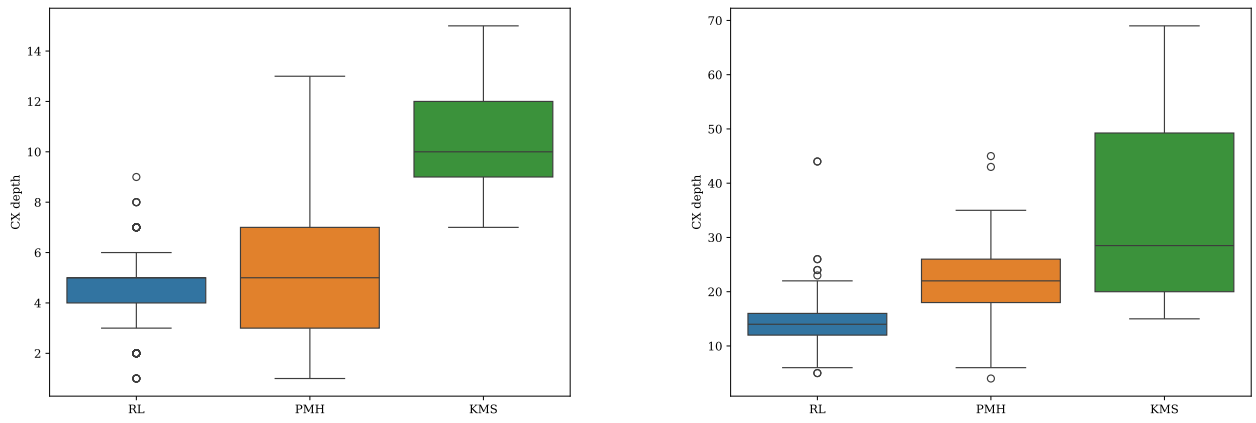


Fig. 4. Comparison between the performance of the proposed RL-based synthesis approach, PMH and KMS using all topologies the models were trained on. In particular, Fig. (a) refers to the 3-qubit model and Fig. (b) refers to the 5-qubit model.

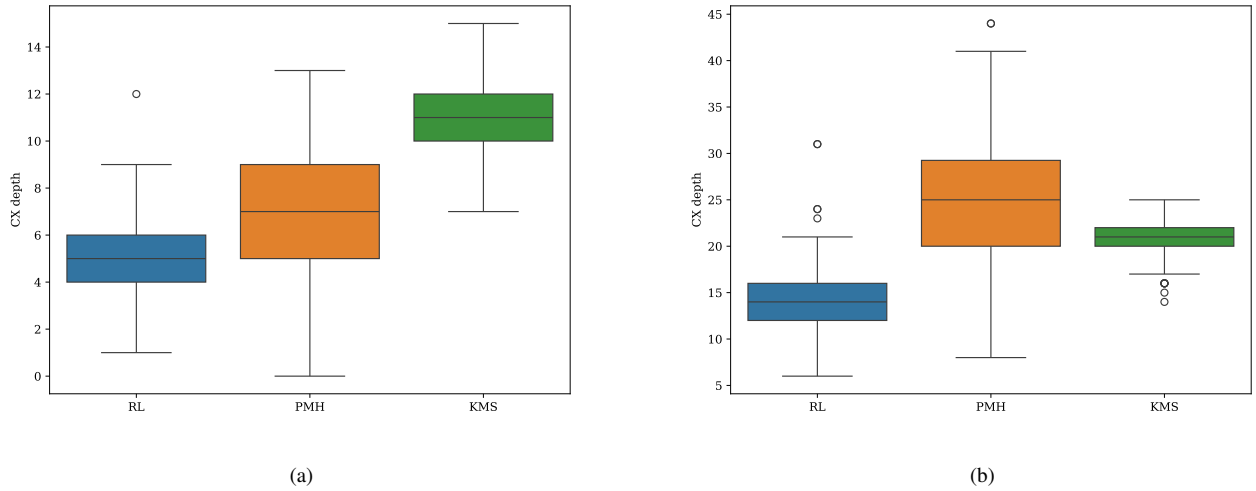


Fig. 5. Comparison between the performance of the proposed RL-based synthesis approach, PMH and KMS using only the linear topology. In particular, Fig. (a) refers to the 3-qubit model and Fig. (b) refers to the 5-qubit model.

3 reports the results obtained in terms of CX depth as box-plots. The boxes represent the range from the first quartile to the third quartile for each set of CX depths. The middle lines indicate the median value of each corresponding set, while the whiskers stretch from the boxes to the minimum and maximum values. Outliers are solutions that deviate significantly from the rest. As it is clear from the plot, RL-A demonstrates better performance compared to RL-B. Specifically, the box related to the distribution of CX depths obtained by exploiting RL-A is entirely under the box related to the RL-B results. Moreover, the average CX depth for RL-A is approximately 14.36, which is notably lower than the 19.86 reported for RL-B. This means that an agent that can choose both the CX and SWAP gates results in synthesized QLRCs with greater CX depth.

C. Performance Evaluation

In this section, the results of the evaluation of the 3-qubit and 5-qubit trained models are discussed and compared to the performance of the two Qiskit synthesizers of QLRCs. In detail, according to the analysis presented in Sec. V-B the best RL-based models are the ones where the action spaces are composed of just CX gates, and the comparison with the other state-of-the-art Qiskit QLRCs synthesizers has been carried out considering just these models. The two Qiskit synthesizers considered are those based on the PMH and the KMS algorithms. To perform the comparison, for both the 3 and 5 qubits scenarios, it has been considered a test dataset composed of 200 operators obtained from Linear

Function circuits with a number of CX and SWAP gates randomly sampled from a range that goes from 1 to 1000. In addition, a topology has been chosen for each operator. The topologies are equally distributed over the dataset in order to test the models on all of them in the same way. The metric used for the comparison is the CX depth of the synthesized circuits, which is intended to be minimized. Figs. 4a and 4b report the CX depth of our RL-based approach, named RL, and the other synthesis approaches for the 3-qubit and 5-qubit cases. It is essential to remember that PMH is able to synthesize Linear Function circuits on all-to-all architectures and KMS works for linear nearest-neighbor architectures. For this reason, a further routing step with Qiskit SDK's SABRE has been applied to circuits resulting from them. Instead, our RL approach provides circuits directly compatible with the given processor topology. Analyzing the comparison, it is possible to observe that our RL-based approach provides synthesized circuits with a smaller CX depth with respect to the other two methods in both cases. In fact, it presents CX depths that cover a less wide range and a lower median than the other approaches. Table I shows the CX depth averages provided by the synthesis approaches. In particular, in the 3-qubit case, our RL outperforms PMH and KMS by 15% and 59%, respectively, while in the 5-qubit case, by 35% and 60%.

Since KMS is known to work only for linear nearest-neighbor architectures, a second phase of testing has been conducted. This allowed for conducting a more in-depth analysis of the performance of the models and confirming the superiority of our RL for all the topologies considered. In particular, Figs. 5a and 5b report another comparison made considering only the linear topology for both 3-qubit and 5-qubit cases. As expected, the median of KMS has decreased, especially in the 5-qubit case, while the medians of RL and PMH have not increased significantly, and RL still remains the one that provides the lowest CX depth values.

Finally, the runtimes of RL did not turn out to be significantly higher compared to those of PMH and KMS. Specifically, for the 3-qubit case, RL required a time of 0.028 ± 0.014 sec, PMH a time of 0.002 ± 0.001 sec, and KMS a time of 0.002 ± 0.001 sec; for the 5-qubit case, the times were 0.089 ± 0.043 sec, 0.002 ± 0.001 sec, and 0.003 ± 0.002 sec for RL, PMH, and KMS, respectively.

TABLE I
TABLE OF THE CX DEPTH AVERAGES OF THE SYNTHESIZED CIRCUITS PROVIDED BY THE SYNTHESIS ALGORITHMS EVALUATED FOR ALL ARCHITECTURES.

Number of qubits	RL	PMH	KMS
3	4.41	5.185	10.695
5	14.165	21.955	35.265

VI. CONCLUSION

This paper presents an RL-based approach for QLRCs that extends the one introduced in [12] by adapting the topology aware approach proposed in previous work [13]. Specifically, through the invalid action masking technique, it removes the need for training a model for each topology of interest. This implies a significant reduction in computational resources and time. The PPO model is capable of synthesizing any Linear Function circuit composed of a considerable number of CX and SWAP gates on a specified architecture. The evaluation session demonstrates superior performance of the proposed approach over existing synthesis methods in terms of CX depth. In the future, the synthesizer presented will be extended to synthesize circuits with more qubits and more complex topologies. Moreover, producing a model that is able to synthesize other classes of circuits composed of non-Clifford gates would be a significant step forward.

These results highlight the potential of RL-driven quantum circuit synthesis as a powerful alternative to traditional heuristic-based techniques, paving the way for more efficient AI-based quantum compilation strategies in the NISQ era.

REFERENCES

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011. [Online]. Available: https://www.amazon.com/Quantum-Computation-Information-10th-Anniversary/dp/1107002176?SubscriptionId=AKIAIOBINVZYXZQZ2U3A&tag=chimbori05-20&linkCode=xm2&camp=2025&creative=165953&creativeASIN=1107002176

[2] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. S. L. Brandao, D. A. Buell, B. Burkett, Y. Chen, Z. Chen, B. Chiaro, R. Collins, W. Courtney, A. Dunsworth, E. Farhi, B. Foxen, A. Fowler, C. Gidney, M. Giustina, R. Graff, K. Guerin, S. Habegger, M. P. Harrigan, M. J. Hartmann, A. Ho, M. Hoffmann, T. Huang, T. S. Humble, S. V. Isakov, E. Jeffrey, Z. Jiang, D. Kafri, K. Kechedzhi, J. Kelly, P. V. Klimov, S. Knysh, A. Korotkov, F. Kostritsa, D. Landhuis, M. Lindmark, E. Lucero, D. Lyakh, S. Mandrà, J. R. McClean, M. McEwen, A. Megrant, X. Mi, K. Michielsen, M. Mohseni, J. Mutus, O. Naaman, M. Neeley, C. Neill, M. Y. Niu, E. Ostby, A. Petukhov, J. C. Platt, C. Quintana, E. G. Rieffel, P. Roushan, N. C. Rubin, D. Sank, K. J. Satzinger, V. Smelyanskiy, K. J. Sung, M. D. Trevithick, A. Vainsencher, B. Villalonga, T. White, Z. J. Yao, P. Yeh, A. Zalcman, H. Neven, and J. M. Martinis, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019. [Online]. Available: https://doi.org/10.1038/s41586-019-1666-5

[3] N. Herrmann, D. Arya, M. W. Doherty, A. Mingare, J. C. Pillay, F. Preis, and S. Prestel, "Quantum utility – definition and assessment of a practical quantum advantage," in *2023 IEEE International Conference on Quantum Software (QSW)*, 2023, pp. 162–174.

[4] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, I. D. Kivlichan, T. Menke, B. Peropadre, N. P. D. Sawaya, S. Sim, L. Veis, and A. Aspuru-Guzik, "Quantum chemistry in the age of quantum computing," *Chemical Reviews*, vol. 119, no. 19, pp. 10 856–10 915, 2019, pMID: 31469277. [Online]. Available: https://doi.org/10.1021/acs.chemrev.8b00803

[5] B. Bauer, S. Bravyi, M. Motta, and G. K.-L. Chan, "Quantum algorithms for quantum chemistry and quantum materials science," *Chemical Reviews*, vol. 120, no. 22, pp. 12 685–12 717, 2020, pMID: 33090772. [Online]. Available: https://doi.org/10.1021/acs.chemrev.9b00829

[6] A. Baiardi, M. Christandl, and M. Reiher, "Quantum computing for molecular biology**," *ChemBioChem*, vol. 24, no. 13, Jun. 2023. [Online]. Available: http://dx.doi.org/10.1002/cbic.202300120

[7] F. P. Maria Schuld, *Supervised Learning with Quantum Computers*, 1st ed. Springer Cham, 2018.

[8] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, Aug. 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79

[9] S. Aaronson and D. Gottesman, "Improved simulation of stabilizer circuits," *Phys. Rev. A*, vol. 70, p. 052328, Nov 2004. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.70.052328

[10] K. N. Patel, I. L. Markov, and J. P. Hayes, "Optimal synthesis of linear reversible circuits," *Quant. Inf. Comput.*, vol. 8, no. 3-4, pp. 0282–0294, 2008.

[11] D. Gottesman, "The heisenberg representation of quantum computers," 1998. [Online]. Available: https://arxiv.org/abs/quant-ph/9807006

[12] D. Kremer, V. Villar, H. Paik, I. Duran, I. Faro, and J. Cruz-Benito, "Practical and efficient quantum circuit synthesis and transpiling with reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2405.13196

[13] G. Acampora, A. Cuzzocrea, M. Lapegna, R. Schiattarella, and A. Vitiello, "Generalizing reinforcement learning-based quantum circuit synthesis across multiple topologies," *Under publication in proceedings of IEEE SSCI 2025*, 2025.

[14] W. Ding, Z. Ming, G. Wang, and Y. Yan, "System-of-systems approach to spatio-temporal crowdsourcing design using improved ppo algorithm based on an invalid action masking," *Knowledge-Based Systems*, vol. 285, p. 111381, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705124000169

[15] M. G. Davis, E. Smith, A. Tudor, K. Sen, I. Siddiqi, and C. Iancu, "Towards optimal topology aware quantum circuit synthesis," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2020, pp. 223–234.

[16] E. Smith, M. G. Davis, J. Larson, E. Younis, L. B. Oftelie, W. Lavrijsen, and C. Iancu, "Leap: Scaling numerical optimization based synthesis using an incremental approach," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, feb 2023. [Online]. Available: https://doi.org/10.1145/3548693

[17] M. Weiden, J. Kalloor, J. Kubiatowicz, E. Younis, and C. Iancu, "Wide quantum circuit optimization with topology aware synthesis," in *2022 IEEE/ACM Third International Workshop on Quantum Computing Software (QCS)*, 2022, pp. 1–11.

[18] B. Rubinstein, "Evolving quantum circuits using genetic programming," in *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*, vol. 1, 2001, pp. 144–151 vol. 1.

[19] M. Weiden, E. Younis, J. Kalloor, J. Kubiatowicz, and C. Iancu, "Improving quantum circuit synthesis with machine learning," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 1–11. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/QCE57702.2023.00093

[20] M. Ostaszewski, L. M. Trenkwalder, W. Masarczyk, E. Scerri, and V. Dunjko, "Reinforcement learning for optimization of variational quantum circuit architectures," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 18 182–18 194. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/9724412729185d53a2e3e7f889d9f057-Paper.pdf

[21] A. Kundu, A. Sarkar, and A. Sadhu, "KANQAS: Kolmogorov-Arnold Network for Quantum Architecture Search," *EPJ Quantum Technology*, vol. 11, no. 1, p. 76, Nov. 2024. [Online]. Available: https://doi.org/10.1140/epjqt/s40507-024-00289-z

[22] X. Zhu and X. Hou, "Quantum architecture search via truly proximal policy optimization," *Scientific Reports*, vol. 13, no. 1, p. 5157, Mar. 2023. [Online]. Available: https://doi.org/10.1038/s41598-023-32349-2

[23] E.-J. Kuo, Y.-L. L. Fang, and S. Y.-C. Chen, "Quantum architecture search via deep reinforcement learning," 2021. [Online]. Available: https://arxiv.org/abs/2104.07715

[24] M. S. Alam, N. F. Berthusen, and P. P. Orth, "Quantum logic gate synthesis as a Markov decision process," *npj Quantum Information*, vol. 9, no. 1, p. 108, Oct. 2023. [Online]. Available: https://doi.org/10.1038/s41534-023-00766-w

[25] L. Moro, M. G. A. Paris, M. Restelli, and E. Prati, "Quantum compiling by deep reinforcement learning," *Communications Physics*, vol. 4, no. 1, p. 178, Aug. 2021. [Online]. Available: https://doi.org/10.1038/s42005-021-00684-3

[26] Y.-H. Zhang, P.-L. Zheng, Y. Zhang, and D.-L. Deng, "Topological quantum compiling with reinforcement learning," *Phys. Rev. Lett.*, vol. 125, p. 170501, Oct 2020. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.125.170501

[27] Q. Chen, Y. Du, Y. Jiao, X. Lu, X. Wu, and Q. Zhao, "Efficient and practical quantum compiler towards multi-qubit systems with deep reinforcement learning*," *Quantum Science and Technology*, vol. 9, no. 4, p. 045002, jul 2024. [Online]. Available: https://dx.doi.org/10.1088/2058-9565/ad420a

[28] T. Gabor, M. Zorn, and C. Linnhoff-Popien, "The applicability of reinforcement learning for the automatic generation of state preparation circuits," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2196–2204. [Online]. Available: https://doi.org/10.1145/3520304.3534039

[29] M. Kölle, T. Schubert, P. Altmann, M. Zorn, J. Stein, and C. Linnhoff-Popien, "A reinforcement learning environment for directed quantum circuit synthesis," 2024. [Online]. Available: https://arxiv.org/abs/2401.07054

[30] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer, "Unitary synthesis of clifford+t circuits with reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2404.14865

[31] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, "Quantum circuit optimization with deep reinforcement learning," 2021. [Online]. Available: https://arxiv.org/abs/2103.07585

[32] Z. T. Wang, Q. Chen, Y. Du, Z. H. Yang, X. Cai, K. Huang, J. Zhang, K. Xu, J. Du, Y. Li, Y. Jiao, X. Wu, W. Liu, X. Lu, H. Xu, Y. Jin, R. Wang, H. Yu, and S. P. Zhao, "Quantum compiling with reinforcement learning on a superconducting processor," 2024. [Online]. Available: https://arxiv.org/abs/2406.12195

[33] D. L. Maslov and M. Roetteler, "Shorter stabilizer circuits via bruhat decomposition and quantum circuit transformations," *IEEE Transactions on Information Theory*, vol. 64, pp. 4729–4738, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:22092011

[34] S. Bravyi and D. Maslov, "Hadamard-free circuits expose the structure of the clifford group," *IEEE Transactions on Information Theory*, vol. 67, no. 7, p. 4546–4563, Jul. 2021. [Online]. Available: http://dx.doi.org/10.1109/TIT.2021.3081415

[35] S. Bravyi, R. Shaydulin, S. Hu, and D. Maslov, "Clifford Circuit Optimization with Templates and Symbolic Pauli Gates," *Quantum*, vol. 5, p. 580, Nov. 2021. [Online]. Available: https://doi.org/10.22331/q-2021-11-16-580

[36] V. V. Shende, A. K. Prasad, I. L. Markov, and J. P. Hayes, "Synthesis of reversible logic circuits," *Trans. Comp.-Aided Des. Integ. Cir. Sys.*, vol. 22, no. 6, p. 710–722, Nov. 2006. [Online]. Available: https://doi.org/10.1109/TCAD.2003.811448

[37] S. A. Kutin, D. P. Moulton, and L. M. Smithline, "Computation at a distance," 2007. [Online]. Available: https://arxiv.org/abs/quant-ph/0701194

[38] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1001–1014. [Online]. Available: https://doi.org/10.1145/3297858.3304023

[39] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[40] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html